BASI DI DATI

DBMS nell’era dei Big Data e dell’AI

NoSQL, distribuzione e convergenza dei modelli

Daniele Riboni · Università degli Studi di Cagliari

Dipartimento di Matematica e Informatica

BASI DI DATI

Obiettivi

1. Capire perché sono nati i database NoSQL e quali limiti del modello relazionale affrontano
2. Conoscere le quattro categorie classiche e i criteri per scegliere tra loro
3. Comprendere i compromessi tra consistenza, disponibilità e latenza (CAP, PACELC)
4. Avere uno sguardo su come il campo si sta evolvendo oggi: NewSQL, multi-modello, database vettoriali

BASI DI DATI

Big Data e i limiti del modello relazionale relazionale

L'esplosione dei Big Data: le 3V

L'esplosione dei **“Big Data”**: dataset massivi e complessi.

Volume	**Velocità**	**Varietà**
quantità di dati	rapidità di generazione	eterogeneità dei formati

I DBMS relazionali non sono pensati per i Big Data

Esempio 1: Dati da sensori in una smart city

1. **Volume dei dati:** ordine di 10^6 dati al minuto (miliardi al giorno)
2. **Dati:** UserID, timestamp, dato dipendente dal sensore (temperatura, presenza, azione, evento...)
3. **Compiti:** rilevare sequenze di dati/eventi; analizzare storici di dati ed estrarre statistiche



Esempio 2: Messaggi su social network

1. **Volume:** Migliaia di messaggi al secondo — centinaia di milioni al giorno — centinaia di miliardi all'anno

2. **Record di un messaggio:** UserID, timestamp, testo, immagini, informazioni aggiuntive...

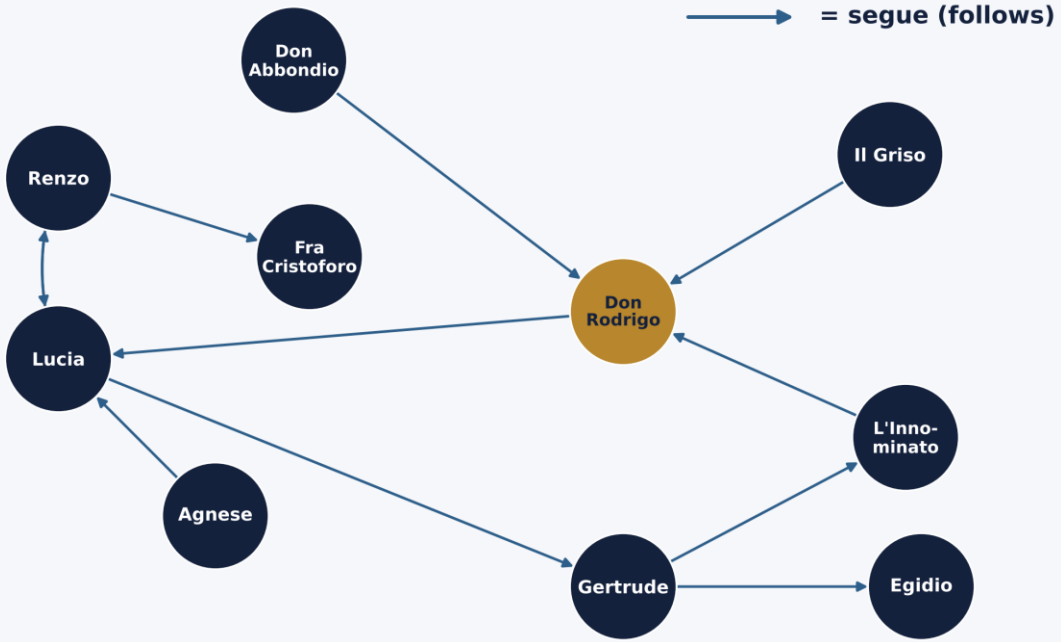
3. **Compiti:** aggiungere un messaggio; modificarne lo stato; recuperare un messaggio dato un criterio di ricerca, hashtag, ecc.



Esempio 3: Connessioni in un social network

1. **Record di amicizia, follower:** `<UserID1, UserID2>`
2. **Record utente:** UserID, nome, foto, profilo ...
3. **Compito:** es. trovare tutti i follower di un follower di un follower ... di un dato utente

Esempio: trovare tutti i follower (a cascata) di Don Rodrigo



Perché il modello relazionale non è ottimale

- Il modello relazionale si basa su valori e tipi di dato → i Big Data sono eterogenei e flessibili
- Le interrogazioni si basano su JOIN → troppo costose
- Le interrogazioni hanno sintassi rigida → i Big Data richiedono interrogazioni “stile Google”
- Grande impegno per affidabilità e concorrenza → non sempre cruciale per le applicazioni Big Data

Requisiti per la gestione dei Big Data

- Evitare complessità non necessaria per ottenere scalabilità
- Schemi semplici (o nessuno schema)
- Sacrificare l'affidabilità in cambio di prestazioni
- Strutture dati più vicine alla programmazione che alla progettazione di basi di dati

“Più codice, meno progettazione del DB”

BASI DI DATI

I sistemi NoSQL

NOSQL

Che cos'è NoSQL

Una famiglia di DBMS che non utilizza il modello relazionale - Non si basano su tabelle

Usano modelli logici diversi:

- coppie `<chiave, valore>`
- documenti (JSON, XML, altri formati...)
- Grafi
- Colonne

NOSQL · TERMINOLOGIA

Perché NoSQL non significa “no SQL”

NoSQL è l'abbreviazione storica di **“Not only SQL”**: non è un rifiuto di SQL, ma di alcuni dei vincoli del modello relazionale “classico”

- Il nome descrive una famiglia di modelli di dati, non un linguaggio di interrogazione
- Molti sistemi NoSQL offrono comunque linguaggi SQL-like (lo vedremo più avanti)
- La distinzione vera è tra il modello relazionale a tabelle fisse e modelli più flessibili:

  chiave-valore, colonna, documento, grafo

GARANZIE NOSQL · CATEGORIE

ACID vs BASE

Approccio BASE: rilassare le proprietà ACID

1. **Basically Available:** il DBMS funziona “praticamente” sempre
2. **Soft state:** non deve essere consistente in ogni istante
3. **Eventually consistent:** il sistema diventa infine consistente quando gli aggiornamenti si propagano

NOSQL · CATEGORIE

Le quattro categorie di DBMS NoSQL

1

Key-value store

coppie <chiave, valore>

2

Wide-Column store

famiglie di colonne per riga

3

Document store

documenti semistrutturati

4

Graph database

nodi, archi e proprietà

Key-value store

- Modello dati: coppie <chiave, valore> (più un timestamp)
- I valori possono contenere qualsiasi tipo di dato (nessuno schema)
- Nessuna interrogazione sui valori: solo sulle chiavi

Key-value store in-memory

Tutti i dati in memoria (a volte con durabilità opzionale)

Key-value store residenti su disco

I dati sono scritti stabilmente su disco

Casi d'uso

- Memorizzare informazioni di sessione
- Profili utente, preferenze: userID univoco, preferenze come lingua, colore, fuso orario...
- Dati del carrello: informazioni di acquisto in formato personalizzato, associate alla chiave userID
- Risultati intermedi di task computazionalmente intensivi (specialmente negli store in-memory)

Facebook *usa Memcached per algoritmi ad alta intensità di dati;* ***LinkedIn*** *usa Venice per gestire dati derivati (suggerimenti, offerte...)*

NOSQL · WIDE-COLUMN STORE

Wide-Column store

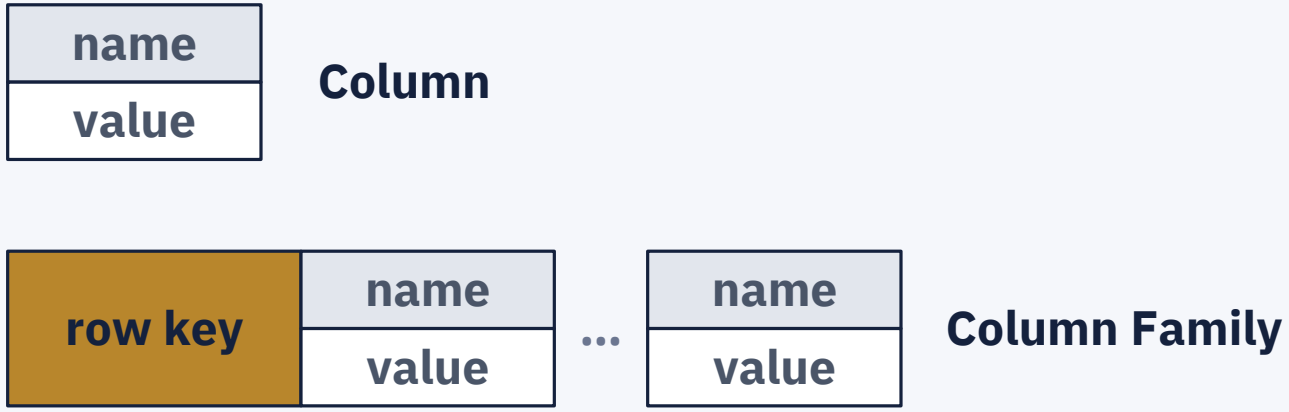
La maggior parte deriva da **Google BigTable:**

- Ogni riga ha diverse “column family”
- Le **column family** sono insiemi di coppie <chiave, valore> con timestamp
- Righe diverse possono avere column family diverse

Nei DBMS relazionali i dati sono normalmente salvati **riga per riga**; nei wide-column store sono salvati **colonna per colonna**. È una differenza fisica, con un impatto enorme sulle prestazioni.

NOSQL · WIDE-COLUMN STORE

I mattoni del modello



- **Schema-less all'interno della famiglia:** righe diverse possono avere colonne diverse
- **Dati sparsi:** se una colonna non ha un valore per una riga, non viene salvata (niente NULL)
- **Timestamped:** ogni valore porta una versione, utile per risolvere i conflitti nei sistemi distribuiti

Esempio completo: due righe, due strutture

In un catalogo prodotti ogni riga ha solo le column family — e il numero di colonne — che le servono: non c'è uno schema comune imposto a tutte.

prodotto:101			
anagrafica			
titolo	autore	anno	prezzo
"1984"	"Orwell"	"1949"	"9.90"
libro			
pagine	genere		
"328"	"Distopico"		

prodotto:450	
anagrafica	
titolo	prezzo
"Penna BIC"	"1.20"

(nient'altro — fine della riga)

Due righe, due strutture: *il libro ha due column family, una con quattro colonne e una con due; la penna ne ha una sola, con due. In un column store questo non è un'eccezione: è la norma.*

Casi d'uso

- Dati di serie temporali (time-series)
- Dati da sensori
- Analisi dei social media

Nella maggior parte dei wide-column store NoSQL (es. Cassandra, HBase), le aggregazioni (es. SUM o AVG) sono assenti o limitate: in genere bisogna farle a livello applicativo

I column store relazionali per l'analisi dei dati (es. ClickHouse, Amazon Redshift, Google BigQuery) invece sono progettati proprio per eseguire aggregazioni in modo molto efficiente, con il normale SQL

NOSQL · DOCUMENT STORE

Document store

- Modello dati: coppie <chiave, documento>
- Documento: JSON, XML, o altri formati semistrutturati (non relazionali)
- I documenti non hanno uno schema fisso
- Interrogazioni anche sul contenuto dei documenti

NOSQL · DOCUMENT STORE

Esempi di documenti e query

Una coppia <nome, cognome>:

```
{"nome":"Mario", "cognome":"Rossi"}
```

Un array di dipendenti in JSON:

```
{"dipendenti":[
  {"nome":"Mario", "cognome":"Rossi"},
  {"nome":"Anna", "cognome":"Bianchi"},
  {"nome":"Piero", "cognome":"Verdi"}
]}
```

Una query aggregata in MongoDB:

```
db.restaurants.aggregate( [
        { $match: { "cuisine":
                        "Brazilian" } },
        { $group: { "_id": "$borough" ,
                "count": { $sum: 1 } } },
        { $match: { "count": {$gt: 5} }
} ] );
```

DOCUMENT STORE

Da tabelle a documento

Uno schema relazionale classico:

```
Clienti(ID, Nome, Cognome, Email)
Ordini(ID, ClienteID → Clienti.ID, Data, Totale)
```

- Come si progetta un documento JSON che rappresenti un cliente insieme ai suoi ordini, sfruttando la possibilità di annidare dati in un document store — niente più JOIN?
- Quali vantaggi e svantaggi introduce questa scelta rispetto alle due tabelle separate?

ESERCIZIO 2 · DOCUMENT STORE

Soluzione: il documento JSON

```
{
  "clienteId": "C001", "nome": "Mario", "cognome": "Rossi",
  "email": "mario.rossi@email.it",
  "ordini": [
    {"ordineId": "O1001", "data": "2026-03-02", "totale": 59.90},
    {"ordineId": "O1002", "data": "2026-04-18", "totale": 124.50}
  ]
}
```

Il JOIN relazionale diventa annidamento: una sola lettura recupera cliente e ordini insieme.
Svantaggio: i documenti possono diventare molto grandi; gli stessi dati possono essere duplicati in più documenti, un cambiamento richiederebbe un aggiornamento in ciascuno di essi

NOSQL · DOCUMENT STORE

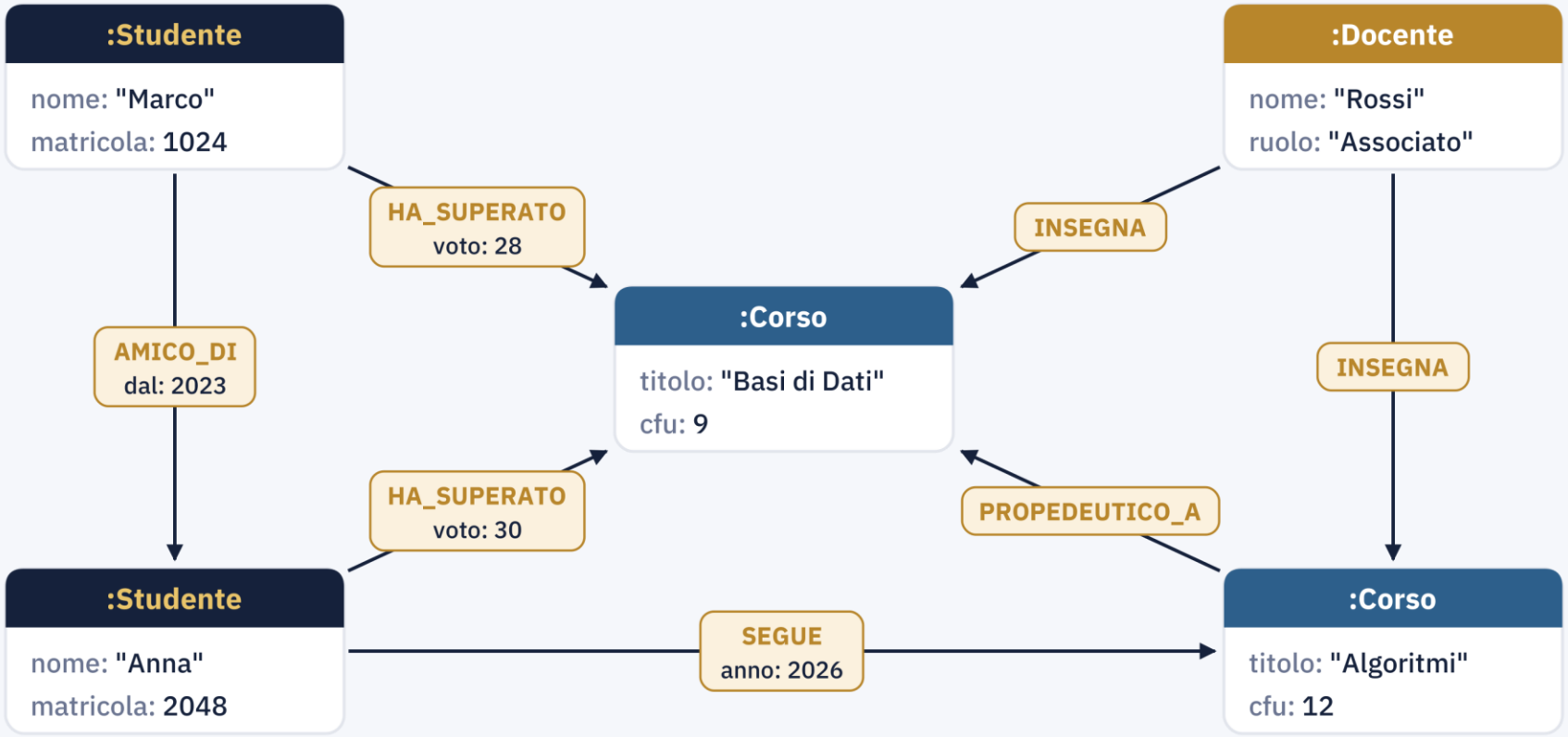
Casi d'uso

- Dati semistrutturati come i post di un blog: hanno alcuni campi fissi e molte voci variabili
- Registrazione di eventi (event logging)
- Web analytics
- Analisi in tempo reale
- Applicazioni e-commerce

NOSQL · GRAPH DATABASE

Graph database

Modello dati: grafi, i cui nodi e archi hanno proprietà rappresentate come coppie chiave-valore



Interrogazione di esempio

Query: *“Quali amici di Marco hanno superato un corso tenuto dal docente Rossi, e con quale voto?”*

```
MATCH (m:Studente {nome: "Marco"})-[:AMICO_DI]->(a:Studente)
      -[s:HA_SUPERATO]->(c:Corso)<-[:INSEGNA]
      -(d:Docente {nome: "Rossi"})
RETURN a.nome AS amico, c.titolo AS corso, s.voto AS voto
```

In Cypher (il linguaggio di Neo4j)

```
+-----------------------------------+
| amico   | corso           | voto  |
+-----------------------------------+
| "Anna" | "Basi di Dati" | 30    |
+-----------------------------------+
```

La stessa query in SQL…

Query: *“Quali amici di Marco hanno superato un corso tenuto dal docente Rossi, e con quale voto?”*

```
SELECT S2.nome AS amico, C.titolo AS corso, E.voto
FROM Studente S1
JOIN Amicizia A ON A.studente = S1.matricola
JOIN Studente S2 ON S2.matricola = A.studente2
JOIN Esame E ON E.studente = S2.matricola
JOIN Corso C ON C.codice = E.corso
JOIN Insegnamento I ON I.corso = C.codice
JOIN Docente D ON D.codice = I.docente
WHERE S1.nome = 'Marco' AND D.nome = 'Rossi';
```

Casi d'uso

Tutte le applicazioni che si basano su qualche tipo di grafo:

- Social network
- Reti biologiche
- Reti narrative
- Analisi dei collegamenti (link analysis)

NOSQL · CATEGORIE

Dagli esempi alle categorie

Riprendiamo i tre esempi della Sezione 2: ora possiamo scegliere il modello più adatto per ciascuno.

Esempio	Categoria più adatta	Perché
Sensori smart city	**Wide-Column store**	scritture ad altissimo volume, serie temporali
Messaggi social	**Document / key-value**	record semistrutturati, accesso quasi sempre per chiave
Grafo social network	**Graph database**	la query stessa è una ricerca nel grafo

NOSQL · CATEGORIE

Panorama dei prodotti

Categoria	Prodotti principali
Key-value store	Redis, Memcached, Riak, Amazon DynamoDB
Wide-Column store	Apache Cassandra, Apache HBase, Google BigTable
Document store	MongoDB, Apache CouchDB, Amazon DocumentDB
Graph database	Neo4j, JanusGraph, Amazon Neptune

NOSQL · CATEGORIE

Le quattro categorie a confronto

Categoria	Modello	Caso d'uso ideale
Key-value	coppie chiave-valore, nessuno schema	sessioni, cache, carrelli
Wide-Column	famiglie di colonne, derivato da BigTable	serie temporali, dati da sensori
Document	documenti JSON/XML, schema flessibile	contenuti semistrutturati, cataloghi
Graph	nodi e archi con proprietà	relazioni complesse, raccomandazioni

ESERCIZIO 3 · LE QUATTRO CATEGORIE

Scegli la categoria giusta

Per ciascuno scenario, scegli la categoria NoSQL più adatta e motiva la scelta:

1. Catalogo prodotti e-commerce con attributi molto variabili da categoria a categoria
2. Cache di sessione per un sito web ad alto traffico
3. Dati da sensori IoT campionati ogni secondo, analizzati come serie temporale
4. Rete di amicizie, per suggerire “persone che potresti conoscere”

ESERCIZIO 3 · LE QUATTRO CATEGORIE

Soluzione e discussione

Scenario	Categoria	Perché
Catalogo e-commerce	**Document**	schema flessibile per attributi variabili
Cache di sessione	**Key-value**	accesso immediato per chiave di sessione
Dati IoT al secondo	**Wide-Column**	scritture ad alto volume, serie temporali
Rete di amicizie	**Graph**	query di traversata sulle relazioni

BASI DI DATI

DBMS distribuiti

DBMS · SCALABILITÀ

Scalabilità verticale vs orizzontale

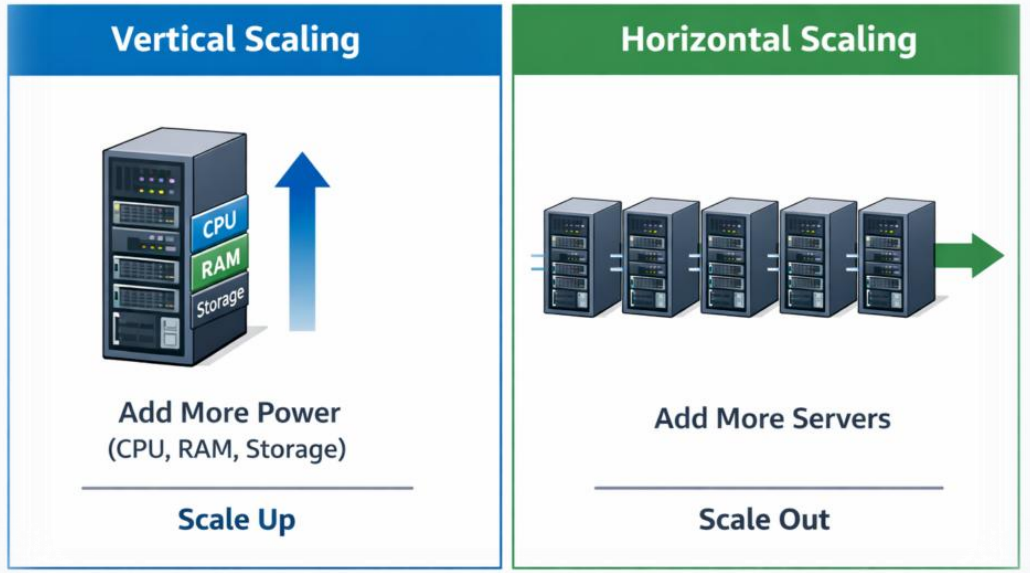
Scalabilità verticale

Aggiungere più potenza (CPU, RAM, ecc.) alla macchina esistente

La scalabilità è limitata

Scalabilità orizzontale

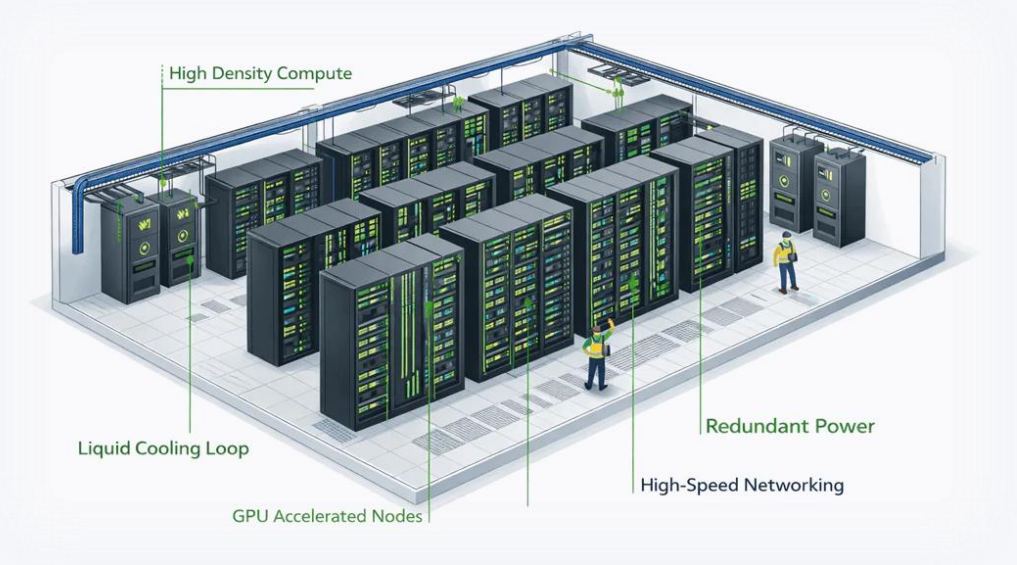
Aggiungere più macchine al sistema, tramite distribuzione e parallelismo



DBMS · SCALABILITÀ

Scalabilità orizzontale

Si distribuisce il carico su migliaia di macchine nei propri data center, anziché potenziare singole macchine



Migliaia di server economici, organizzati in rack, lavorano in parallelo in ogni data center

Ma è difficile garantire al contempo consistenza e affidabilità:
lo vedremo tra poco con il teorema CAP

DBMS DISTRIBUITI · TEOREMA CAP

Proprietà CAP dei sistemi distribuiti

- **C** **Consistency (Consistenza):** in ogni istante, tutti i nodi vedono gli stessi dati
- **A** **Availability (Disponibilità):** il sistema risponde sempre alle richieste
- **P** **Partition tolerance (Tolleranza alle partizioni):** il sistema continua a funzionare anche se la comunicazione tra i server è inaffidabile, o in caso di guasto di un nodo

Teorema CAP

> *“Non è possibile soddisfare contemporaneamente tutte e tre le proprietà CAP in un sistema distribuito”*
> **— Eric Brewer, 1999**

- Se il DBMS deve essere tollerante alle partizioni (P), **bisogna scegliere tra C e A**
- C può essere sostituita da **“eventual consistency”**: tutte le repliche convergeranno allo stesso stato dopo che la connettività di rete è stata ripristinata

Teorema CAP: interpretazione

Immaginiamo una base di dati S partizionata in due insiemi di nodi S1, S2 che non possono comunicare tra loro.



- Se nega tutte le richieste di scrittura sia in S1 che in S2: **è consistente ma non disponibile**
- Se S1 e S2 accettano richieste di scrittura: **S è disponibile ma possibilmente inconsistente**

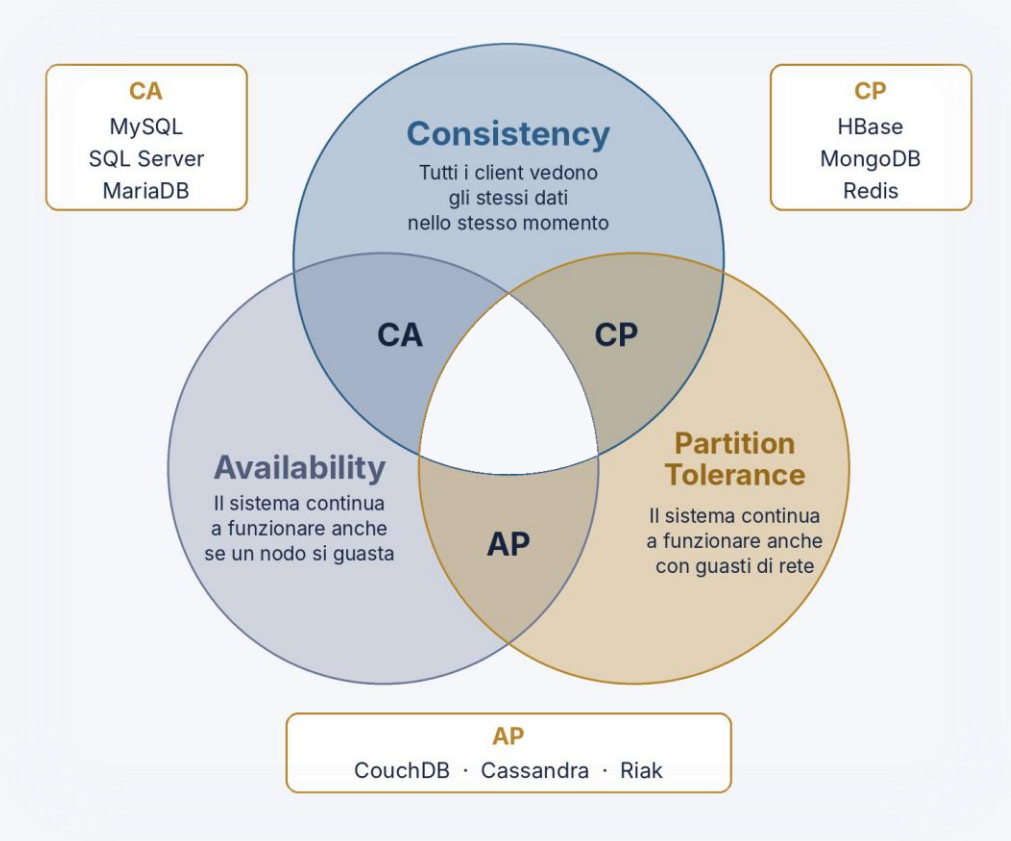
Il compromesso CAP

Ogni sistema distribuito sceglie due proprietà su 3

AP: una replica eroga servizio anche in una partizione, ma può essere inconsistente

CP: sempre consistente, anche in una partizione, ma una replica può negare il servizio

CA: disponibile e consistente, a meno che non vi sia una partizione



Oltre CAP: il teorema PACELC

Il teorema PACELC (Daniel Abadi, 2010) estende CAP: anche senza partizioni, un sistema distribuito deve scegliere tra latenza e consistenza.

- **P**artition: se c'è una partizione, si sceglie tra **A**vailability e **C**onsistency — come nel teorema CAP
- **E**lse: anche in assenza di partizioni, si sceglie tra bassa **L**atenza e **C**onsistenza forte

Classificazione	Significato	Esempi
PA/EL	in partizione sceglie A, altrimenti bassa latenza	Cassandra, DynamoDB
PC/EC	in partizione sceglie C, altrimenti consistenza forte	HBase, MongoDB (config. tipica)

ESERCIZIO 1 · CAP E PACELC

Un trade-off da risolvere

Una banca gestisce conti correnti su due data center, a Milano e a Francoforte, che replicano i dati in tempo reale. Un guasto di rete interrompe la comunicazione fra i due per alcuni minuti (partizione).

- **1.** Durante il guasto, sceglieresti C o A? Motiva pensando alle conseguenze di un prelievo bancario.

- **2.** Anche in condizioni normali (nessuna partizione), quale trade-off PACELC ha senso per un sistema bancario: bassa latenza o consistenza forte? Perché?

ESERCIZIO 1 · CAP E PACELC

Soluzione e discussione

- **Durante la partizione:** **C**. Meglio rifiutare temporaneamente un'operazione che rischiare un doppio prelievo da due data center disallineati. Il sistema è quindi CP.
- **In condizioni normali:** **ancora C**. I sistemi bancari privilegiano quasi sempre la consistenza forte (PC/EC): il costo di un errore sul saldo supera quello di qualche millisecondo di latenza in più.

Contrappunto: un sistema di raccomandazioni e-commerce farebbe la scelta opposta (PA/EL) — meglio mostrare dati leggermente obsoleti che rallentare l'esperienza utente

BASI DI DATI

Limiti, sfide ed evoluzioni del NoSQL

NOSQL · LIMITI

Mancanza di standard nelle query

Grande varietà di linguaggi, nessuno standard — e le interrogazioni non sono portabili tra sistemi NoSQL diversi

Categoria	Linguaggio di interrogazione
Key-value store	nessuno (put, get, update, delete)
Wide-Column store	programmatico, o SQL-like (Apache Hive, Pig Latin...)
Document store	proprietario, orientati a documenti
Graph database	Cypher, altri linguaggi proprietari

NOSQL · LIMITI

Consistenza e affidabilità

Se il DBMS non garantisce la consistenza, deve farlo l'applicazione

- Lo sforzo si sposta da “SQL” a “programmazione”
- Gestire conflitti e consistenza eventuale ricade sul codice applicativo, con maggiore complessità e rischio di bug
- I requisiti di business possono cambiare nel tempo: una scelta adeguata oggi potrebbe non bastare domani

Complessità operativa

Adottare NoSQL spesso significa gestire più sistemi specializzati invece di un unico DBMS.

- **Polyglot persistence:** un'applicazione può usare key-value per le sessioni, document per il catalogo, graph per le raccomandazioni...
- Più sistemi da installare, monitorare, aggiornare e proteggere
- Dati duplicati o sincronizzati tra sistemi diversi: nuovi rischi di inconsistenza

NOSQL · LIMITI

Scalabilità vs funzionalità

I NoSQL scalano orizzontalmente, ma spesso sacrificano query complesse e transazioni multi-record

- Anche i DBMS relazionali possono scalare: **i DBMS main-memory tengono tutti i dati in RAM**
- Non esiste una soluzione universalmente migliore: la scelta dipende dai requisiti specifici dell'applicazione

NewSQL: scalabilità con ACID e SQL

I sistemi NewSQL cercano il meglio di entrambi i mondi: la scalabilità orizzontale del NoSQL e le transazioni ACID con SQL standard del modello relazionale.

- **Google Spanner:** database globalmente distribuito, consistenza forte tramite orologi atomici e GPS (TrueTime)
- **CockroachDB:** ispirato a Spanner, open source, resiliente al guasto di singoli nodi
- **TiDB:** compatibile con il protocollo MySQL, separa storage e calcolo

HTAP: unificare OLTP e OLAP

HTAP (Hybrid Transactional/Analytical Processing): eseguire transazioni operative e analisi su grandi volumi senza spostare i dati tra sistemi separati.

- Tradizionalmente: un DBMS OLTP (Online Transaction Processing) per le transazioni, un data warehouse separato per l'analisi (Online Analytical Processing, OLAP), con scambio dati tra i due
- **HTAP elimina lo scambio:** stesso sistema, dati sempre aggiornati per entrambi gli usi
- Esempi: SAP HANA, TiDB, SingleStore

Data Warehouse: sistema che analizza i dati per individuare tendenze, elaborare statistiche e supportare le decisioni; es. quali prodotti hanno venduto maggiormente negli ultimi 12 mesi? quali sono le variazioni delle vendite tra estate e inverno?

SQL su NoSQL

Molti sistemi NoSQL offrono comunque un livello SQL-like, per abbassare la curva di apprendimento.

- **CQL** (Cassandra Query Language): sintassi simile a SQL per interrogare Cassandra
- **Apache Phoenix:** motore SQL sopra HBase, compila le query in scansioni native
- Risultato: usabilità di SQL, con i limiti del modello NoSQL sottostante — niente JOIN arbitrari, transazioni multi-riga spesso limitate

NOSQL · EVOLUZIONI RECENTI

Database ibridi multi-modello

I confini tra le categorie classiche si stanno sfumando: un singolo prodotto supporta più modelli di dati nativamente

- **PostgreSQL con JSONB:** colonne relazionali e documenti JSON nello stesso database
- **MongoDB:** oggi offre anche ricerca testuale, vettoriale e geografica, oltre al modello a documenti
- **ArangoDB, Azure Cosmos DB:** documento, grafo e chiave-valore nello stesso motore

Vantaggio: meno sistemi da gestire — si attenua la complessità operativa

CASO D’USO

NewSQL o NoSQL classico?

Una fintech deve gestire pagamenti tra milioni di utenti nel mondo: ogni transazione deve essere atomica e consistente (niente doppio addebito), ma il sistema deve scalare orizzontalmente su più data center e restare disponibile 24/7.

- Un key-value store basterebbe da solo?
- Quale caratteristica, tipica del NewSQL, è indispensabile in questo scenario?

CASO D’USO

Soluzione e discussione

Un key-value store puro non basta: garantisce disponibilità e scalabilità, ma non transazioni ACID multi-record (addebitare A e accreditare B in modo atomico)

Serve un sistema NewSQL come Spanner o CockroachDB: transazioni distribuite ACID mantenendo la scalabilità orizzontale — esattamente il compromesso che i NoSQL classici avevano sacrificato.

BASI DI DATI

Uno sguardo al futuro (e al presente)

AI E VETTORI

L'era dell'AI generativa: nuove esigenze

I modelli di AI generativa rappresentano testo, immagini e altri dati come **embedding**: vettori numerici ad alta dimensionalità che catturano il significato

- Cercare per significato, non per uguaglianza: trovare i contenuti “più simili” a una query, non quelli che la contengono alla lettera
- I DBMS tradizionali — relazionali e NoSQL classici — non sono ottimizzati per calcolare similarità tra milioni di vettori
- Nasce una nuova categoria: **i database vettoriali**

Documento	Embedding (semplificato)
«Come cucinare la pasta»	`[0.21, 0.85, 0.32]`
«Ricetta per gli spaghetti»	`[0.19, 0.82, 0.35]`
«Motore di un'automobile»	`[0.91, 0.12, 0.67]`

AI E VETTORI

Database vettoriali e RAG

Un database vettoriale indicizza embedding e risponde a query di similarità (magari approssimata) in millisecondi, anche su miliardi di vettori (su architetture cloud)

- Prodotti: Pinecone, Weaviate, Qdrant, Milvus
- **RAG (Retrieval-Augmented Generation):** prima di rispondere, un LLM recupera i documenti più rilevanti da un database vettoriale (o alter fonti) e li usa come contesto
- È oggi il modo più comune per dare a un LLM accesso a conoscenza aggiornata o privata, senza riaddestrarlo

AI E VETTORI

Integrazione vettoriale nei NoSQL esistenti

Invece di aggiungere un sistema separato, molti database esistenti hanno integrato la ricerca vettoriale

- **MongoDB Atlas Vector Search:** indici vettoriali nello stesso database documentale
- **DynamoDB, Azure Cosmos DB:** ricerca vettoriale nativa nei rispettivi servizi gestiti
- **PostgreSQL con pgvector:** colonne vettoriali accanto a tabelle relazionali

Un altro esempio di convergenza multi-modello

AI E DATABASE

Uno sguardo al futuro

Due tendenze da tenere d'occhio, ancora in evoluzione:

- **Interfacce in linguaggio naturale:** strumenti che traducono domande libere in query SQL o NoSQL, abbassando la barriera d'accesso ai dati
- **Database autonomi e self-tuning:** sistemi che regolano automaticamente indici, partizionamento e risorse in base al carico

Promettenti, ma ancora da maturare: restano strumenti di supporto, non sostituti della progettazione

INFRASTRUTTURA

Database serverless

Un database serverless scala automaticamente (anche a zero) e fattura in base all'uso effettivo, senza gestione manuale di server o cluster.

- Ideale per carichi imprevedibili: applicazioni event-driven, agenti software, prototipi
- Esempi: DynamoDB on-demand, Azure Cosmos DB serverless, Google Firestore

Svantaggio: meno controllo fine su prestazioni e costi rispetto a un cluster dedicato

INFRASTRUTTURA

Cloud-native e multi-cloud

- La maggior parte dei NoSQL moderni è pensata per girare nativamente su infrastrutture cloud (Kubernetes, storage a oggetti, reti distribuite)
- **Multi-cloud:** distribuire gli stessi dati su più provider per resilienza o requisiti normativi
- **Rischio di vendor lock-in:** le API proprietarie di ogni cloud rendono costoso cambiare provider

Le alternative open source (a partire da Kubernetes stesso) riducono ma non eliminano questo rischio

INFRASTRUTTURA

Un cenno all'edge computing

Alcuni database girano direttamente sul dispositivo (smartphone, sensore, auto), sincronizzandosi col cloud quando possibile

- **Couchbase Lite, SQLite con estensioni di sync:** database embedded per mobile e IoT
- La sfida principale è la **conflict resolution**: cosa succede se lo stesso dato viene modificato offline su due dispositivi diversi?

PUBLISH/SUBSCRIBE

Il paradigma publish/subscribe

1. **Publisher:** produttori di informazioni (es. quotazioni di borsa, feed di notizie...)
2. **Subscriber:** consumatori di informazioni
3. **Broker:** identificano gli eventi che corrispondono agli interessi dei consumatori

Middleware publish/subscribe: centralizzato vs distribuito, persistente (DB) vs transitorio, basato su topic vs su contenuto, affidabile vs best-effort, ecc.

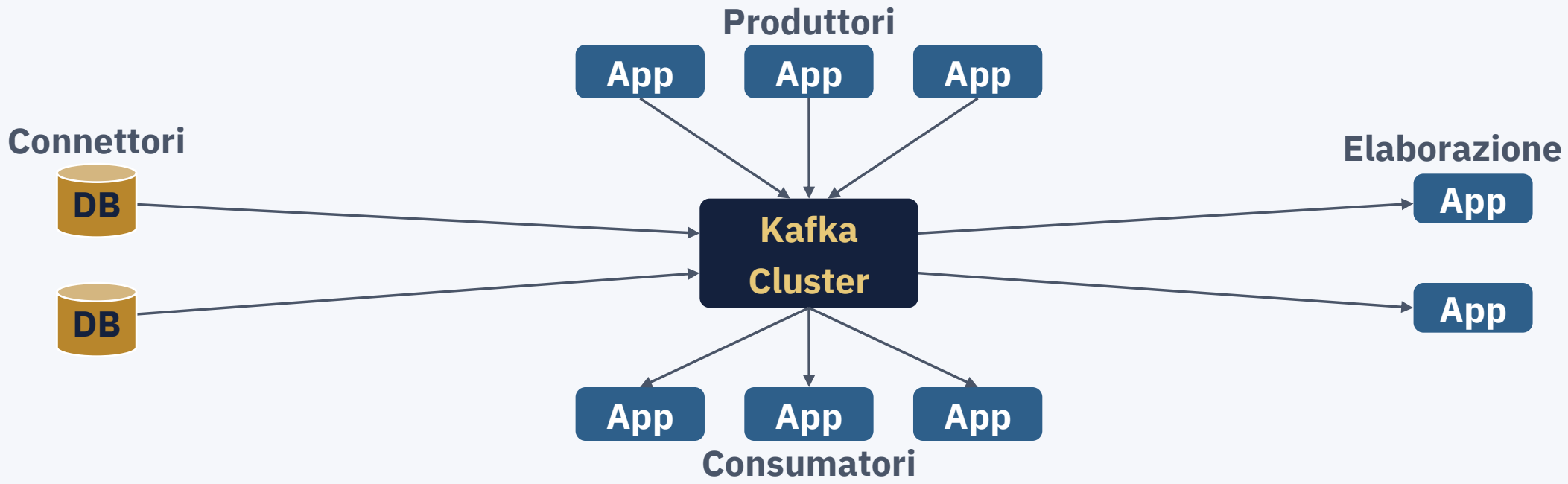
PUBLISH/SUBSCRIBE

Vantaggi del publish/subscribe

- **Spazio:** produttori e consumatori non devono conoscersi a vicenda
- **Tempo:** produttori e consumatori non devono partecipare nello stesso momento
- **Sincronizzazione:** i produttori non vengono bloccati, i consumatori ricevono i messaggi in modo asincrono
- Indirizzamento indiretto, basato su dati e sottoscrizioni

`publish()` `subscribe()` `unsubscribe()` `notify()`

Publish/subscribe su larga scala



DATA LAKE

Data lake: cos'è

Un **data lake** è un repository centralizzato che conserva dati grezzi, in qualunque formato e a qualunque scala, così come arrivano, insieme a metadata, catalogazione, controllo degli accessi

1. **Schema-on-read, non schema-on-write:** la struttura si applica quando i dati vengono letti e analizzati, non quando vengono scritti
2. **Dati eterogenei nel formato nativo:** log, JSON, CSV, immagini, video... fianco a fianco, senza trasformazioni preventive
3. **Storage a oggetti, economico e scalabile:** Amazon S3, Azure Data Lake Storage, ...

DATA LAKE

Data lake vs data warehouse

I due termini si confondono spesso, ma rispondono a esigenze diverse.

	Data lake	Data warehouse
Dati	**grezzi, eterogenei**	**strutturati, puliti**
Schema	**on-read**	**on-write**
Scopo tipico	**data science, ML, esplorazione**	**business intelligence, reportistica**
Utenti	**data engineer, data scientist**	**analista di business**

Evoluzione: il **Data Lakehouse**, che cerca di combinare i vantaggi dei Data Lake con quelli dei Data Warehouse *(Databricks, Snowflake, Dremio...)*

CONCLUSIONI

Dal relazionale, al NoSQL puro, ai sistemi ibridi

Il panorama è cambiato: dalle quattro categorie NoSQL “pure” di un decennio fa, siamo arrivati a un ecosistema di sistemi ibridi e specializzati.

- **NewSQL** per chi ha bisogno di ACID e scala insieme
- **Multi-modello** per ridurre la complessità operativa del polyglot persistence
- **Database vettoriali** per le applicazioni basate su AI

CONCLUSIONI

Il futuro è la convergenza

- I confini tra “relazionale” e “NoSQL” si stanno sfumando
- La domanda non è più “quale categoria?”, ma “quali garanzie servono a questa applicazione, e quale sistema le offre al minor costo operativo?”

Conoscere i modelli di dati e i loro trade-off resta più utile che conoscere un prodotto specifico: i prodotti cambiano, i princìpi — CAP, PACELC, ACID/BASE — restano.

CONCLUSIONI

Relazionale o NoSQL? Il criterio pratico

Non esiste una risposta univoca: la scelta dipende da come l'applicazione userà i dati.

Se l'applicazione richiede…	Scegli…
dati strutturati, relazioni complesse, transazioni multi-tabella	**DBMS relazionale**
scala massiccia, schema flessibile, accesso per chiave singola	**NoSQL classico**
ACID e scala orizzontale insieme	**NewSQL**
ricerca per similarità su embedding	**Database vettoriale**

CONCLUSIONI

Sfide aperte

- **Standardizzazione:** ancora nessun linguaggio di interrogazione universale per NoSQL
- **Portabilità:** migrare da un sistema NoSQL a un altro spesso richiede riscrivere la logica applicativa
- **Sicurezza e governance:** dati distribuiti su più sistemi e cloud, più superfici da proteggere
- **Complessità crescente:** un'architettura sempre più eterogenea richiede competenze più ampie